

```

"""
io_utils.py
=====

Entrada/salida de datos: carga del dataset de panel y tipificacion de columnas.

Responsabilidad unica: dejar en memoria un ``DataFrame`` limpio de tipos y una
clasificacion explicita de cada columna (numerica / categorica / temporal /
booleana). Ninguna decision de seleccion se toma aqui.
"""

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger

LOGGER = obtener_logger("io")

class ErrorCargaDatos(RuntimeError):
    """Fallo al leer o interpretar el dataset."""

# -----
# Carga
# -----
def cargar_dataset(cfg: ConfigPipeline) -> pd.DataFrame:
    """Lee el dataset segun su extension.

    Formatos soportados: ``.csv`` / ``.txt``, ``.parquet``, ``.feather``,
    ``.xlsx`` / ``.xls``, ``.pkl``.

    Raises
    -----
    ErrorCargaDatos
        Si el archivo no existe, el formato no se soporta o la lectura falla.
    """
    ruta = Path(cfg.ruta_dataset)
    if not ruta.is_file():
        raise ErrorCargaDatos(f"No existe el archivo de datos: '{ruta.resolve()}'.")

    sufijo = ruta.suffix.lower()
    LOGGER.info("Cargando dataset '%s' (formato %s)...", ruta, sufijo or "?")

    try:
        if sufijo in (".csv", ".txt"):
            df = pd.read_csv(ruta, sep=cfg.csv_sep, encoding=cfg.csv_encoding, low_memory=False)
        elif sufijo == ".parquet":
            df = pd.read_parquet(ruta)
        elif sufijo == ".feather":
            df = pd.read_feather(ruta)
        elif sufijo in (".xlsx", ".xls"):
            df = pd.read_excel(ruta)
        elif sufijo == ".pkl":
            df = pd.read_pickle(ruta)
        else:
            raise ErrorCargaDatos(f"Extension no soportada: '{sufijo}'.")
    except ErrorCargaDatos:
        raise
    except Exception as exc: # noqa: BLE001
        raise ErrorCargaDatos(f"No se pudo leer '{ruta}': {exc}") from exc

    if df.empty:
        raise ErrorCargaDatos(f"El dataset '{ruta}' se leyo pero no tiene filas.")

    LOGGER.info("Dataset cargado: %d filas x %d columnas.", df.shape[0], df.shape[1])

```

```

return df

# -----
# Tipificacion
# -----
def inferir_tipo(serie: pd.Series) -> str:
    """Clasifica una columna en una de las familias que usa el pipeline.

    Devuelve uno de: ``NUMERICA``, ``BOOLEANA``, ``FECHA``, ``CATEGORICA``.

    Notas
    ----
    - ``bool`` se separa de ``numerica`` porque su varianza y sus percentiles
      se interpretan distinto (es una proporcion, no una magnitud).
    - Una columna de texto que en realidad contiene numeros guardados como
      string se intenta convertir; si >=95% de los valores no nulos se
      convierten sin error, se trata como numerica. Esto evita descartar
      variables validas por un problema de formato del origen.
    """
    if pd.api.types.is_bool_dtype(serie):
        return "BOOLEANA"
    if pd.api.types.is_datetime64_any_dtype(serie):
        return "FECHA"
    if pd.api.types.is_numeric_dtype(serie):
        return "NUMERICA"

    no_nulos = serie.dropna()
    if len(no_nulos) > 0:
        convertidos = pd.to_numeric(no_nulos, errors="coerce")
        if convertidos.notna().mean() >= 0.95:
            return "NUMERICA"
    return "CATEGORICA"

def tipificar_dataset(
    df: pd.DataFrame, cfg: ConfigPipeline
) -> tuple[pd.DataFrame, dict[str, str]]:
    """Normaliza tipos y devuelve el mapa columna -> tipo inferido.

    Acciones:
    - Convierte a numerico las columnas de texto que en realidad son numeros.
    - Intenta parsear la columna de tiempo como fecha; si no lo logra, la deja
      como esta (un panel puede indexarse por entero: 1..T, 202401, etc.).
    - Fuerza el target a numerico cuando es binario textual ("SI"/"NO", "1"/"0").

    Se registran todas las conversiones aplicadas para trazabilidad.
    """
    df = df.copy()
    tipos: dict[str, str] = {}
    conversiones: list[str] = []

    for col in df.columns:
        tipo = inferir_tipo(df[col])

        # Texto que en realidad es numero -> se convierte y se documenta.
        if tipo == "NUMERICA" and not pd.api.types.is_numeric_dtype(df[col]):
            df[col] = pd.to_numeric(df[col], errors="coerce")
            conversiones.append(f"{col}: texto -> numerico")

        tipos[col] = tipo

    # --- Columna de tiempo: se intenta interpretar como fecha -----
    col_t = cfg.columna_tiempo
    if col_t in df.columns and tipos.get(col_t) == "CATEGORICA":
        parseada = pd.to_datetime(df[col_t], errors="coerce", format="mixed")
        if parseada.notna().mean() >= 0.95:
            df[col_t] = parseada
            tipos[col_t] = "FECHA"
            conversiones.append(f"{col_t}: texto -> fecha")

```

```

# --- Target: si es categorico binario, se mapea a 0/1 -----
col_y = cfg.columna_target
if col_y in df.columns and tipos.get(col_y) == "CATEGORICA":
    valores = sorted(df[col_y].dropna().astype(str).str.upper().unique())
    mapeos = [
        ({"NO", "SI"}, {"NO": 0, "SI": 1}),
        ({"N", "S"}, {"N": 0, "S": 1}),
        ({"NO", "YES"}, {"NO": 0, "YES": 1}),
        ({"FALSE", "TRUE"}, {"FALSE": 0, "TRUE": 1}),
        ({"BAD", "GOOD"}, {"GOOD": 0, "BAD": 1}),
    ]
    for esperado, mapa in mapeos:
        if set(valores) == esperado:
            df[col_y] = df[col_y].astype(str).str.upper().map(mapa)
            tipos[col_y] = "NUMERICA"
            conversiones.append(f"{col_y}: categorico binario -> 0/1 ({mapa})")
            break

if conversiones:
    LOGGER.info("Conversiones de tipo aplicadas (%d):", len(conversiones))
    for c in conversiones:
        LOGGER.info("    - %s", c)
else:
    LOGGER.info("No fue necesaria ninguna conversion de tipos.")

resumen = pd.Series(tipos).value_counts().to_dict()
LOGGER.info("Distribucion de tipos inferidos: %s", resumen)
return df, tipos

def detectar_tipo_target(y: pd.Series) -> str:
    """Determina si el target es ``BINARIO``, ``MULTICLASE`` o ``CONTINUO``.

    La distincion importa porque cambia como se calculan Gini e IV:

    - BINARIO    -> Gini = 2*AUC-1 y WOE/IV clasicos.
    - CONTINUO   -> el target se discretiza en cuantiles para el IV y el Gini
                     se aproxima con la correlacion de rangos (Spearman).
    - MULTICLASE -> se reduce al esquema "clase mayoritaria vs resto" y se
                     documenta la simplificacion.

    """
    y = y.dropna()
    n_unicos = y.nunique()

    if n_unicos <= 1:
        return "DEGENERADO"
    if n_unicos == 2:
        return "BINARIO"
    if not pd.api.types.is_numeric_dtype(y):
        return "MULTICLASE"
    # Enteros con pocos niveles distintos -> tratamiento de clases.
    es_entero = np.allclose(y.dropna() % 1, 0)
    if es_entero and n_unicos <= 10:
        return "MULTICLASE"
    return "CONTINUO"

```